



MD اريد ملف ال

Odoo ERP Expert Team Skill

تعريف الملف 1.

مدارات ERP احترافي قادر على تصميم وتنفيذ وتشغيل حلول Odoo هذا الملف هو مرجع رسمي لبناء فريق تطوير System configuration وSecurity. [web:1] وOdoo Developer documentation **:مصدر إضافي**

الغاية من المهارة 2.

الهدف الرئيسي 2.1

Odoo: تمكين الفريق من بناء حلول

- صحيحة من الناحية الوظيفية .
 - آمنة من الناحية التقنية .
 - قابلة للتوسع والصيانة .
 - قابلة للترقية بين الإصدارات .
 - قابلة للربط مع أنظمة خارجية .
 - مناسبة للمؤسسات متعددة الشركات والمواقع والقنوات .
 - متوافقة مع السياسات التنظيمية والحوكمة الداخلية .
- Odoo Developer + Security + Testing. [web:1][web:16][web:135] **:مصدر إضافي**

مبدأ التصميم 2.2

:بشكل احترافي عبر "كتابة كود فقط"، بل عبر منظومة تشمل Odoo لا يُبنى

- تحليل الأعمال .
 - تصميم البيانات .
 - ضبط الأمن .
 - تحسين الأداء .
 - اختبار التوافق .
 - التوثيق .
 - المراقبة بعد الإطلاق .
 - ISO.مواءمة الحل مع الحوكمة وال
- Testing + System configuration + Developer docs. [web:135][web:133][w **:مصدر إضافي**

دور المهارة داخل الفريق 3.

دورها 3.1

:فردية"، بل إطار عملي لبناء فريق كامل قادر على Skill هذه المهارة ليست مجرد

- من البداية للنهاية Odoo تنفيذ مشاريع .
- تقسيم المسؤوليات بوضوح .
- ضمان جودة التسليم .
- توحيد أسلوب العمل .
- تقليل المخاطر التقنية .
- دعم وضبط العمليات التشغيلية داخل الشركات .

****مصدر إضافي:**** Developer docs + Testing. [web:1][web:135]

كيف تُستخدم 3.2

- كمرجع توظيف.
- كمرجع تدريب.
- كمرجع مراجعة كود.
- كمرجع تصميم معماري.
- كمرجع تسليم مشاريع.
- كمرجع لحوكمة العمليات والاعتمادات والتصعيدات.

****مصدر إضافي:**** Developer + Security. [web:1][web:16]

هيكل الفريق 4.

4.1 Solution Architect

مسؤول عن:

- فهم المتطلبات.
- رسم المعمارية العامة.
- تحديد حدود التخصيص.
- اختيار التكاملات المناسبة.
- وضع خطة الترقية والمخاطر.
- مواءمة الحل مع سياسات الشركة والحوكمة.

****مصدر إضافي:**** Developer + Upgrade scripts. [web:1][web:92]

4.2 Odoo Backend Developer

مسؤول عن:

- models.
- ORM.
- workflows.
- business rules.
- access logic.
- server actions.
- scheduled jobs.
- migration support.

****مصدر إضافي:**** ORM + Testing. [web:2][web:135]

4.3 Odoo Frontend Developer

مسؤول عن:

- OWL components.
- client actions.
- widgets.
- view customization.
- assets.
- responsive interface.

****مصدر إضافي:**** Owl components + Tutorials. [web:9][web:93]

4.4 Odoo Website/Portal Developer

مسؤول عن:

- website pages.
- themes.
- snippets.
- portal layouts.
- public components.
- customer experience.

****مصدر إضافي:**** Owl on portal/website + Website docs. [web:89][web:101]

4.5 API & Integration Developer

مسؤول عن:

- JSON-2.
- controllers.
- external systems.
- webhooks.
- sync jobs.
- authentication.

****مصدر إضافي:**** External JSON-2 API + HTTP controllers. [web:73][web:81]

4.6 Database / Performance Engineer

مسؤول عن:

- PostgreSQL.
- indexes.
- query plans.
- vacuum.
- monitoring.
- storage strategy.

****مصدر إضافي:**** ORM + System configuration. [web:2][web:133]

4.7 QA / Test Engineer

مسؤول عن:

- unit tests.
- integration tests.
- regression tests.
- UAT support.
- release validation.

****مصدر إضافي:**** Testing Odoo. [web:135]

4.8 Technical Lead

مسؤول عن:

- code review.
- technical decisions.
- standards.
- security.
- delivery readiness.
- mentoring.

****مصدر إضافي:**** Developer + Testing + Security. [web:1][web:135][web:16]

4.9 DevOps / Release Engineer

مسؤول عن:

- environments.
- deployments.
- backups.
- rollback.
- CI/CD.
- logs and observability.
- server hardening.

****مصدر إضافي:**** System configuration + Testing. [web:133][web:135]

4.10 Business Process Owner

مسؤول عن:

- تعريف سير العمل.
- اعتماد النماذج التشغيلية.
- تحديد نقاط الموافقة.

- تحديد الاستثناءات.
- مواءمة النظام مع سياسة الشركة.
- التأكيد من توافق العمليات مع الحوكمة و ISO.
- **مصدر إضافي** Restrict access to data + Security. [web:122][web:16]

4.11 Governance / Compliance Owner

مسؤول عن:

- ضبط السياسات.
- توثيق الإجراءات.
- التأكد من الفصل بين المهام.
- مراقبة الاعتمادات.
- متابعة التتبع والأثر التدقيقي.
- التأكد من أن الحل لا يكسر إجراءات المراجعة الداخلية.
- **مصدر إضافي** Security + System configuration. [web:16][web:133]

5. المجالات التقنية الأساسية

5.1 Backend Development

ما يجب إتقانه

- ORM recordsets.
- relations.
- computed fields.
- constraints.
- inheritance.
- wizards.
- server actions.
- cron jobs.
- workflow methods.
- **مصدر إضافي** ORM API + Testing. [web:2][web:135]

قواعد مهمة

- متى أمكن SQL بدل ORM استخدم.
- compute مباشرة داخل self لا تعدل.
- كمنطق خادمي onchange لا تعتمد على.
- عند الحاجة للبحث أو التقارير `store=True` استخدم.
- للحقول المفلتره بكثرة `index=True` أصف.
- **مصدر إضافي** ORM API + Security. [web:2][web:16]

مثال مهني

```
```python
```

```
from odoo import models, fields, api, _
```

```
from odoo.exceptions import ValidationError
```

```
class ProjectTask(models.Model):
```

```
 _name = 'project.task'
```

```
 _description = 'Task'
```

```
 name = fields.Char(required=True, index=True)
```

```
 amount = fields.Float()
```

```
 total = fields.Float(compute='_compute_total', store=True)
```

```
 @api.depends('amount')
```

```
 def _compute_total(self):
```

```
 for record in self:
```

```

 record.total = record.amount * 1.15

@api.constrains('amount')
def _check_amount(self):
 for record in self:
 if record.amount < 0:
 raise ValidationError(_('Amount cannot be negative'))

```

مصدر إضافي: ORM + Testing. [web:2][web:135]

## 5.2 Frontend Development

ما يجب إتقانه

- OWL components.
  - templates.
  - services.
  - state and props.
  - registry.
  - client actions.
  - custom widgets.
- مصدر إضافي: Owl components + Tutorials. [web:9][web:93]

قواعد مهمة

- اجعل المكونات صغيرة وقابلة لإعادة الاستخدام.
  - افصل البيانات عن العرض.
  - مع البيانات غير الموثوقة t-raw بدل t-esc استخدم.
  - والجوال RTL اجعل التصميم متوافقاً مع.
- مصدر إضافي: Owl docs + Security. [web:9][web:16]

على الموقع والبوابة OWL

public\_components عبر website وال portal على Owl components يوضح طريقة استخدام Odoo 19 registry و web.assets\_frontend و <owl-component> tags. [web:89][web:9]

مصدر إضافي: Owl portal/website. [web:89]

## 5.3 Website and Portal

## Website

- صفحات تعريفية.
  - صفحات خدمات.
  - landing pages.
  - forms.
  - themes.
  - snippets.
- Website docs + Tutorials. [web:101][web:14] مصدر إضافي

## Portal

- dashboard.
  - orders.
  - invoices.
  - documents.
  - tickets.
  - profile.
  - secure downloads.
- Owl on portal/website + Security. [web:89][web:16] مصدر إضافي

## قواعد مهمة

- عند صفحات العملاء website layouts بدل portal layouts استخدم.
  - record rules راع الصلاحيات وال.
  - لا تُظهر بيانات حساسة إلا للمستخدم المخول.
  - ابن تجربة استخدام بسيطة وواضحة.
- Security + Restrict access to data. [web:16][web:122] مصدر إضافي

## 5.4 APIs and Integration

### النوع الحديث

access ويطبق bearer API key مع <model>/<method> عبر External JSON-2 API يوفر Odoo 19 rights وrecord rules وfield accesses والأمان الداخلي بنفس نموذج الأمان.

Reference: Odoo External JSON-2 API

JSON-2 API + Security. [web:73][web:16] مصدر إضافي

## ما يجب إتقانه

- API keys.
  - authentication.
  - versioned endpoints.
  - validation.
  - pagination.
  - error handling.
  - idempotency.
  - retries.
  - webhooks.
  - audit logs.
- JSON-2 API + HTTP controllers. [web:73][web:81] مصدر إضافي

## قواعد مهمة

- عامة بدون حماية endpoints لا تكتب.
  - لا تعرض أكثر مما هو لازم.
  - مناسبًا JSON-2 إذا كان XML-RPC/new code لا تعتمد على.
  - اجعل التكامل قابلاً للمراقبة والاسترداد.
- JSON-2 API + Security. [web:73][web:16] مصدر إضافي

## JSON-2 API – Python مثال

```
import requests

BASE_URL = "https://mycompany.example.com/json/2"
API_KEY = "YOUR_API_KEY"
DATABASE = "mycompany"

headers = {
 "Authorization": f"bearer {API_KEY}",
 "X-Odoo-Database": DATABASE,
 "Content-Type": "application/json",
}

res = requests.post(
 f"{BASE_URL}/res.partner/search_read",
 headers=headers,
 json={
 "domain": [["is_company", "=", True]],
 "fields": ["name", "email", "phone"],
 "limit": 10,
 "order": "name asc",
 },
)
```

```
res.raise_for_status()
data = res.json()
```

مصدر إضافي: JSON-2 API. [web:73]

## مثال Controller — JSON API

```
from odoo import http
from odoo.http import request
from odoo.exceptions import AccessError

class ApiController(http.Controller):

 @http.route('/api/v1/partners', type='json', auth='user', csrf=False, methods=['GET'])
 def partners(self, domain=None, fields=None, limit=20):
 if not request.env.user.has_group('base.group_user'):
 raise AccessError("Not allowed")

 domain = domain or []
 fields = fields or ['name', 'email', 'phone']

 data = request.env['res.partner'].search_read(domain, fields, limit=limit)
 return {'success': True, 'count': len(data), 'data': data}
```

مصدر إضافي: HTTP Controllers + Security. [web:81][web:16]

## 5.5 Database and Storage

### ما يجب إتقانه

- PostgreSQL tuning.
- indexes.
- query plans.
- vacuum/autovacuum.
- locks and transactions.
- attachment storage.
- data retention.

مصدر إضافي: System configuration + ORM. [web:133][web:2]

### قواعد مهمة

- راقب الاستعلامات البطيئة.
- الداخلية queries مع loops تجنب.
- عند الحاجة read\_group و search\_read استخدم.
- بشكل اعتباطي raw SQL لا تعتمد على.



- عند وجود بيانات مكانية PostGIS استخدم .  
ORM + System configuration. [web:2][web:133] مصدر إضافي

## 5.6 Reports

### ما يجب إتقانه

- QWeb PDF.
- QWeb HTML.
- Excel exports.
- branded headers and footers.
- translations.
- multi-company layouts.
- Reports + Developer docs. [web:7][web:1] مصدر إضافي

### قواعد مهمة

- التقرير يجب أن يكون مفهومًا من أول مرة.
- لا تكس البيانات دون ترتيب.
- حافظ على الاتساق البصري.
- راع طبعة الطباعة والهوامش واللغة.
- Reports + Testing. [web:7][web:135] مصدر إضافي

## 5.7 Automation

### أدوات الأتمتة

- server actions.
- automated actions.
- cron jobs.
- scheduled sync.
- email workflows.
- marketing automation عند الحاجة.
- Developer docs + Testing. [web:1][web:135] مصدر إضافي

### قواعد مهمة

- الأتمتة يجب أن تحل مشكلة حقيقية.
- لا تجعلها تخفي منطقًا معقدًا لا يمكن تتبعه.

- واضح logging يجب أن يكون له automation كل .  
[web:16][web:135] Testing + Security: مصدر إضافي

## 5.8 Security

### ما يجب إتقانه

- ACL.
  - groups.
  - record rules.
  - field-level access.
  - safe controllers.
  - safe rendering.
  - safe attachments.
- [web:16][web:122] Security + Restrict access to data: مصدر إضافي

### خطوط حمراء

- SQL injection.
  - XSS.
  - CSRF.
  - unsafe sudo.
  - missing ACL.
  - weak record rules.
  - exposing sensitive records.
  - bypassing ORM without reason.
- [web:16][web:133] Security + System configuration: مصدر إضافي

ORM. هما أساس التحكم، وأن القواعد تُطبّق عبر record rules و access rights تؤكد صراحة أن Odoo  
[web:16][web:122]

[web:16][web:122] Security + Restrict access to data: مصدر إضافي

## 6. الخبرات البيزنس والورك فلو داخل الشركات

### 6.1 فهم العمليات الداخلية

يجب على الفريق فهم:

- دورة الطلب.
- دورة الشراء.
- دورة الاعتماد.

- دورة التوريد.
  - دورة المخزون.
  - دورة الفوترة.
  - دورة التحصيل.
  - دورة المشاريع.
  - دورة الموارد البشرية.
  - دورة خدمة العملاء.
  - دورة الشكاوى والتصعيد.
  - دورة الإغلاق والمراجعة.
- Restrict access to data + Developer docs. [web:122][web:1]  
مصدر إضافي

## 6.2 تصميم الورك فلو

:الورك فلو الناجح يجب أن يجب على

- من ينشئ الطلب؟
  - من يراجع؟
  - من يعتمد؟
  - متى ينتقل للحالة التالية؟
  - ما الشروط؟
  - ما الاستثناءات؟
  - ما سجلات التدقيق؟
  - ما آثار الرفض أو الإرجاع؟
- Security + Testing. [web:16][web:135]  
مصدر إضافي

## 6.3 مستويات الاعتماد

- إنشاء.
  - مراجعة.
  - اعتماد أول.
  - اعتماد ثان.
  - اعتماد استثنائي.
  - تنفيذ.
  - إغلاق.
  - مراجعة لاحقة.
- Security + Developer docs. [web:16][web:1]  
مصدر إضافي

## 6.4 فصل المهام

من منظور الحوكمة، يجب فصل

- من يطلب.
  - من يراجع.
  - من يعتمد.
  - من ينفذ.
  - من يراجع التنفيذ.
  - من يدقق لاحقًا.
- Restrict access to data + Security. [web:122][web:16] مصدر إضافي

## 6.5 التتبع والأثر التدقيقي

كل عملية مهمة يجب أن تترك

- who.
  - what.
  - when.
  - before.
  - after.
  - reason.
  - approval trail.
- Security + System configuration. [web:16][web:133] مصدر إضافي

## 6.6 أمثلة تنظيمية

مقبول

- أمر شراء يمر بمراجعة ثم اعتماد ثم تنفيذ ثم إقفال.
  - طلب خدمة يمر بتصنيف ثم اعتماد ثم تحويل ثم إغلاق.
- Security + Testing. [web:16][web:135] مصدر إضافي

مفضل

- واضحة workflow بناء حالات.
  - ربط كل انتقال بسبب وشروط.
  - تسجيل مبررات الرفض والاستثناء.
- Security + Developer docs. [web:16][web:1] مصدر إضافي

## مرفوض

- تغيير حالة سجل بدون منطق اعتماد.
  - تمرير الطلبات خارج السلسلة الإدارية.
  - إلغاء أثر التدقيق.
- [web:16][web:122] Security + Restrict access to data. :مصدر إضافي

## 7. والحوكمة ISO المواصفة مع 7.

### 7.1 مبادئ عامة

:الخبرة المطلوبة يجب أن تراعي

- التوثيق.
  - ضبط النسخ.
  - الاعتمادات.
  - التتبع.
  - فصل المسؤوليات.
  - إدارة المخاطر.
  - تحسين مستمر.
  - مراجعات دورية.
- [web:1][web:135][web:16] Developer docs + Testing + Security. :مصدر إضافي

### 7.2 متطلبات عملية

- كل عملية حرجية يجب أن تكون موثقة.
  - كل استثناء يجب أن يسجل.
  - كل تغيير يجب أن يكون قابلاً للمراجعة.
  - كل اعتماد يجب أن يكون مرتبطاً بصاحب الصلاحية.
  - كل تعديل مهم يجب أن يترك أثراً تدقيقياً.
  - fallback وowner كل تكامل يجب أن يكون له.
- [web:16][web:133] Security + System configuration. :مصدر إضافي

### 7.3 ضوابط حوكمة

- عدم منح صلاحيات عامة بلا حاجة.
  - عدم خلط الأدوار الحرجة.
  - عدم تجاوز مراحل الموافقة.
  - trace عدم تنفيذ عمليات حساسة دون.
  - عدم الاعتماد على تعليمات شفوية غير موثقة.
- [web:16][web:122] Security + Restrict access to data. :مصدر إضافي

## أمثلة 7.4

### مفضل

```
if not self.env.user.has_group('my_module.group_approver'):
 raise AccessError(_('You are not allowed to approve this document'))
```

مصدر إضافي: Security. [web:16]

### مقبول

```
self.message_post(body=_('Approved by %s') % self.env.user.name)
```

مصدر إضافي: Developer docs + Security. [web:1][web:16]

### مرفوض

```
self.sudo().write({'state': 'approved'})
```

مصدر إضافي: Security. [web:16]

## مستوى الخبرة المطلوب داخل الفريق 8.

### 8.1 Junior

- فهم أساسيات Odoo.
  - تنفيذ موديولات بسيطة.
  - تعديل views.
  - الأساسي ORM استخدام.
- مصدر إضافي: Tutorials + ORM. [web:14][web:2]

### 8.2 Mid-Level

- بناء workflows.
  - كتابة wizards.
  - تطوير APIs.
  - security basics التعامل مع.
  - reports تنفيذ.
- مصدر إضافي: Developer docs + Security + Reports. [web:1][web:16][web:7]

### 8.3 Senior

- architecture. تصميم
  - performance. إدارة
  - OWL.
  - portal and website.
  - migrations.
  - integration design.
  - business operations. فهم
- مصدر إضافي: OWL + Upgrade scripts + Developer docs. [web:9][web:92][web:1]

### 8.4 Lead/Architect

- code. مراجعة
  - التصميم. اعتماد التصميم
  - المخاطر. إدارة المخاطر
  - standards. توحيد
  - التسليم. الإشراف على التسليم
  - ISO. مراجعة توافق الحل مع الحوكمة و
- مصدر إضافي: Security + Testing + System configuration. [web:16][web:135][web:133]

## 9. نظام العمل داخل الفريق.

### 9.1 دورة المشروع

1. جمع المتطلبات.
  2. تحليل الأعمال.
  3. technical design. كتابة
  4. تقسيم المهام.
  5. التنفيذ.
  6. مراجعة الكود.
  7. الاختبار.
  8. الأداء.
  9. الأمان.
  10. التوثيق.
  11. النشر.
  12. المراقبة بعد الإطلاق.
- مصدر إضافي: Testing + System configuration + Developer docs. [web:135][web:133][web:1]

## 9.2 آلية العمل

- owner لها feature كل.
  - .يراجع PR كل
  - checklist له release كل
  - test plan كل تغيير كبير له
  - fallback plan له integration كل
  - approval map له workflow critical كل
- Testing + Security. [web:135][web:16] مصدر إضافي

## 9.3 معايير التسليم

- code clean.
  - docs جاهزة.
  - tests ناجحة.
  - security checked.
  - performance checked.
  - migration path defined.
  - process approved.
  - governance alignment confirmed.
- Testing + System configuration. [web:135][web:133] مصدر إضافي

## 9.4 قاعدة صارمة للعمل

يجب عدم التنفيذ بعشوائية  
إذا كان للحل أكثر من طريقة، يجب

- شرح كل طريقة.
  - توضيح المميزات.
  - توضيح العيوب.
  - توضيح التعارضات مع الموديولات الأخرى إن وجدت.
  - توضيح أثر كل خيار على الأمان والأداء والترقية.
- Testing + Security + Upgrade scripts. [web:135][web:16][web:92] مصدر إضافي

## 9.5 إدارة التحديات

أي تحدٍ تقني أو تعارض وظيفي أو خطر تكاملي يجب

- أو نظام التتبع أولاً بأول GitHub رفعه إلى
- توثيقه.
- ربطه بالموديول أو الإصدار.



- عدم إخفائه داخل الكود أو تجاوزه بصمت.
- Testing + Developer docs. [web:135][web:1] مصدر إضافي

## 10. الجودة والحوكمة التقنية.

### 10.1 Code Review

- أسماء واضحة.
  - منطق مقسّم.
  - duplication لا.
  - side effects خفية لا.
  - bypass للصلاحيات لا.
  - لا كسر لسلسلة الموافقات.
- Security + Testing. [web:16][web:135] مصدر إضافي

### 10.2 Testing

- unit tests.
  - integration tests.
  - functional tests.
  - portal tests.
  - API tests.
  - migration tests.
  - workflow approval tests.
- Testing Odoo. [web:135] مصدر إضافي

### 10.3 Documentation

- document module purpose.
  - document models and fields.
  - document endpoints.
  - document rules.
  - document deployment.
  - document rollback.
  - document business approvals.
- Developer docs + System configuration. [web:1][web:133] مصدر إضافي

## 10.4 Observability

- logs.
- monitoring.
- alerts.
- error tracking.
- performance tracking.
- sync tracking.
- approval tracking.
- exception tracking.

مصدر إضافي: System configuration + Testing. [web:133][web:135]

## 11. Website and Portal Engineering

### 11.1 Website Engineering

- themes.
- snippets.
- layouts.
- pages.
- SEO basics.
- assets.
- responsive behavior.

مصدر إضافي: Website docs + OWL. [web:101][web:9]

### 11.2 Portal Engineering

- public\_components.
- portal dashboards.
- secure visibility.
- customer documents.
- personalized views.
- safe downloads.

مصدر إضافي: Owl on portal/website + Security. [web:89][web:16]

## 11.3 Portal example

```
/** @odoo-module */
import { Component, onMounted, useState } from "@odoo/owl";
import { registry } from "@web/core/registry";

export class GeoMapWidget extends Component {
 setup() {
 this.state = useState({
 loading: true,
 points: [],
 });

 onMounted(async () => {
 const res = await fetch("/api/v1/geo/points", {
 headers: { "Content-Type": "application/json" },
 });
 const data = await res.json();
 this.state.points = data.points || [];
 this.state.loading = false;
 });
 }
}

GeoMapWidget.template = "my_module.GeoMapWidget";
registry.category("public_components").add("my_module.GeoMapWidget", GeoMapWidget);
```

مصدر إضافي: Owl portal/website. [web:89]

## 12. API Engineering

### 12.1 JSON-2 API

Odoo 19 يقدم External JSON-2 API كواجهة HTTP عبر `/json/2/<model>/<method>` مع bearer API key، حسب قاعدة البيانات / doc وتوثيقه متاح في.

Reference: [Odoo External JSON-2 API](#)

مصدر إضافي: JSON-2 API + Security. [web:73][web:16]

### 12.2 Team Rules

- every endpoint versioned.
  - every endpoint logged.
  - every input validated.
  - every output structured.
  - every integration documented.
- مصدر إضافي: JSON-2 API + Testing. [web:73][web:135]

## 12.3 Security Rules

- API key rotation.
  - least privilege.
  - short-lived credentials where possible.
  - separate keys per integration.
  - error sanitization.
  - rate limiting at gateway or application level.
- مصدر إضافي: Security + System configuration. [web:16][web:133]

## 12.4 JSON-2 example

```
import requests

BASE_URL = "https://mycompany.example.com/json/2"
API_KEY = "YOUR_API_KEY"
DATABASE = "mycompany"

headers = {
 "Authorization": f"bearer {API_KEY}",
 "X-Odoo-Database": DATABASE,
 "Content-Type": "application/json",
}

res = requests.post(
 f"{BASE_URL}/res.partner/search_read",
 headers=headers,
 json={
 "domain": [["is_company", "=", True]],
 "fields": ["name", "email", "phone"],
 "limit": 10,
 "order": "name asc",
 },
)
res.raise_for_status()
data = res.json()
```

مصدر إضافي: JSON-2 API. [web:73]

## 12.5 Controller example

```
from odoo import http
from odoo.http import request
from odoo.exceptions import AccessError

class ApiController(http.Controller):

 @http.route('/api/v1/partners', type='json', auth='user', csrf=False, methods=['POST'])
 def partners(self, domain=None, fields=None, limit=20):
 if not request.env.user.has_group('base.group_user'):
```

```
raise AccessError("Not allowed")

domain = domain or []
fields = fields or ['name', 'email', 'phone']

data = request.env['res.partner'].search_read(domain, fields, limit=limit)
return {'success': True, 'count': len(data), 'data': data}
```

مصدر إضافي: HTTP Controllers + Security. [web:81][web:16]

## 13. Database Governance

### 13.1 PostgreSQL

- tuning.
- indexes.
- autovacuum.
- locks.
- maintenance.
- bloat checks.

مصدر إضافي: System configuration + ORM. [web:133][web:2]

### 13.2 Geospatial and Mapping

وربطها من PostGIS إذا كان المشروع يحتوي خرائط أو مواقع أو أصول هندسية، فالأفضل بناء طبقة مكانية عبر حسب الحاجة AutoCAD أو ArcGIS أو QGIS مع الاستفادة من API، أو تكامل Odoo ORM خلال

مصدر إضافي: Developer docs + OCA geospatial ecosystem. [web:1][web:106][web:108]

### 13.3 Data Governance

- retention.
- archiving.
- auditing.
- ownership.
- access classification.

مصدر إضافي: Security + System configuration. [web:16][web:133]

## 14. GIS / CAD Integration

## 14.1 Scope

- ArcGIS.
  - QGIS.
  - PostGIS.
  - AutoCAD.
  - geolocation.
  - spatial layers.
  - map visualization.
  - engineering data exchange.
- مصدر إضافي: OCA geospatial + Odoo geolocation. [web:106][web:108][web:111]

## 14.2 Recommended Architecture

- Odoo ORM for business entities.
  - PostGIS for spatial storage.
  - QGIS for analysis and editing.
  - ArcGIS for enterprise mapping.
  - AutoCAD for engineering drawings.
  - API bridge for synchronization.
- مصدر إضافي: OCA geospatial + Odoo docs. [web:106][web:108][web:111]

## 14.3 ArcGIS references

Reference: [ArcGIS Maps SDK for JavaScript](#) [web:142]

Reference: [ArcGIS Get Started](#) [web:143]

Reference: [ArcGIS Sample Code](#) [web:148]

Reference: [ArcGIS References](#) [web:146]

## 14.4 OCA GIS references

Reference: [OCA/geospatial](#) [web:106][web:108]

Reference: [Leaflet Map View \(OpenStreetMap\)](#) [web:153]

## 14.5 ArcGIS integration example

```
import Map from "@arcgis/core/Map";
import MapView from "@arcgis/core/views/MapView";
import FeatureLayer from "@arcgis/core/layers/FeatureLayer";

const map = new Map({
 basemap: "streets-vector",
});

const view = new MapView({
```

```

 container: "viewDiv",
 map,
 center: [46.6753, 24.7136],
 zoom: 11,
 });

 const layer = new FeatureLayer({
 url: "https://services.arcgis.com/your_service_url",
 });

 map.add(layer);

```

مصدر إضافي: ArcGIS Maps SDK + References. [web:142][web:146]

## 14.6 Odoo + ArcGIS pattern

```

from odoo import models, fields

class GeoAsset(models.Model):
 _name = 'geo.asset'
 _description = 'Geo Asset'

 name = fields.Char(required=True)
 latitude = fields.Float()
 longitude = fields.Float()
 arcgis_object_id = fields.Char(index=True)
 address = fields.Char()

 def action_push_to_arcgis(self):
 for rec in self:
 payload = {
 "name": rec.name,
 "latitude": rec.latitude,
 "longitude": rec.longitude,
 "address": rec.address,
 }
 # requests.post(ARC_GIS_URL, json=payload, headers=...)

```

مصدر إضافي: ArcGIS SDK + Odoo docs. [web:142][web:1]

## 15. Migration and Upgrades

### 15.1 Scope

- versioned migrations.
- upgrade scripts.
- data transformation.
- schema adjustments.
- compatibility checks.

- rollback planning.  
مصدر إضافي: Upgrade scripts + Developer docs. [web:92][web:1]

## 15.2 Team Rules

- every release needs upgrade notes.
- every breaking change needs migration mapping.
- every version bump needs smoke tests.
- every custom integration needs compatibility review.  
مصدر إضافي: Upgrade scripts + Testing. [web:92][web:135]

## 16. Allowed / Accepted / Preferred / Forbidden / Rejected

### 16.1 قاعدة عليا

مباشرة، وتشمل Odoo هذه المهارة مخصصة فقط لكل ما يخدم:

- Backend.
- Frontend.
- Website.
- Portal.
- APIs.
- Reports.
- Database.
- Automation.
- Security.
- Integration.
- GIS/CAD عندما يكون مرتبطاً بـ Odoo.
- Upgrade and migration.
- داخل الشركات Business workflows and governance.

مباشرة. لا يتم تحويل المهارة Odoo أي طلب خارج هذه الحدود يجب عدم التوسع فيه، ويُرد عليه فقط بما يخدم إلى شرح عام خارج النظام، ولا إلى تقنيات غير مرتبطة بالمنصة.  
مصدر إضافي: Developer docs + Security. [web:1][web:16]

### 16.2 Allowed

```
partner = self.env['res.partner'].search([('active', '=', True)])
```

```
@api.depends('amount')
def _compute_total(self):
```



```
for record in self:
 record.total = record.amount * 1.15
```

```
<t t-esc="record.name"/>
```

مصدر إضافي: ORM + Security. [web:2][web:16]

### 16.3 Accepted

```
def action_sync(self):
 self.sudo().write({'synced': True})
```

```
self.env.cr.execute(
 "UPDATE my_table SET state = %s WHERE id = %s",
 ['done', record.id]
)
```

```
<t t-raw="safe_html"/>
```

مصدر إضافي: Security + Developer docs. [web:16][web:1]

### 16.4 Preferred

```
records = self.env['sale.order'].search([('state', '=', 'sale')])
```

```
orders = self.env['sale.order'].read_group(
 [('state', '=', 'sale')],
 ['partner_id', 'amount_total:sum'],
 ['partner_id']
)
```

```
registry.category("public_components").add("my_module.Widget", Widget);
```

```
<t t-esc="partner.name"/>
```

مصدر إضافي: ORM + Owl. [web:2][web:9]

### 16.5 Forbidden

```
self.env.cr.execute("SELECT * FROM res_partner")
```

```
self.sudo().write(vals)
```

```
<div t-row="user_input"/>
```

```
query = "DELETE FROM sale_order WHERE id = " + str(order_id)
self.env.cr.execute(query)
```

مصدر إضافي: Security + System configuration. [web:16][web:133]

## 16.6 Rejected

```
partners = self.env['res.partner'].sudo().search([])
```

```
import os
os.system("rm -rf /tmp/test")
```

```
<t t-row="comment.body"/>
```

```
self.env.cr.execute("UPDATE account_move SET paid = TRUE")
```

مصدر إضافي: Security + Testing. [web:16][web:135]

## 17. قواعد لا يجوز تجاوزها.

### 17.1 الأمن أولاً

- لا SQL injection.
  - لا XSS.
  - غير مضبوط CSRF لا.
  - عشوائي sudo لا.
  - لا فتح بيانات دون قواعد وصول.
- مصدر إضافي: Security + System configuration. [web:16][web:133]

### 17.2 Odoo أولاً

- عندما يكون ممكنًا ORM استخدم.
- core. بدل تعديل inheritance استخدم.
- ACLs و record rules استخدم.
- بالطريقة الرسمية OWL استخدم.

- بشكل مؤمن controllers أو JSON-2 استخدم.
- ORM + Security + Owl + JSON-2. [web:2][web:16][web:9][web:73] مصدر إضافي

### 17.3 قابلية الصيانة

- لا تكتب حلولاً "تشتغل الآن" وتكسر الترقية.
- لا تُدخل منطقاً غامضاً.
- template أو view داخل business logic لا تضع.
- لا يمكن اختياره feature لا تبين.
- Testing + Upgrade scripts. [web:135][web:92] مصدر إضافي

### 17.4 الالتزام بنطاق المهارة

مباشرة يجب عدم توسيعه داخل الملف Odoo أي موضوع لا يخدم:  
الملف يجب أن يبقى:

- مختصراً في نطاقه.
- Odoo عميقاً في.
- صارماً في حدوده.
- واضحاً في المسموح والممنوع.
- Developer docs + Security. [web:1][web:16] مصدر إضافي

## 18. تأمين السيرفرات DevOps

### 18.1 التشغيل الآمن

في بيئة إنتاج، يجب Odoo عند نشر:

- دائماً HTTPS استخدام.
- reverse proxy خلف Odoo تشغيل.
- منع الوصول غير المصرح لصفحات إدارة قواعد البيانات.
- استخدام كلمات مرور قوية وفريدة.
- في البيئات العامة demo data تعطيل.
- عند الحاجة list\_db = False أو db\_filter استخدام.
- PostgreSQL user غير superuser جعل.
- database عمل نسخ احتياطي يومي للـ filestore.
- عزل العملاء أو البيئات عند تعدد المستخدمين.
- System configuration. [web:133] مصدر إضافي

## 18.2 الممارسات التشغيلية

- تحديثات منتظمة.
- timeouts وال workers مراقبة.
- CPU ضبط حدود الذاكرة وال.
- filestore حماية ال.
- مراقبة السجلات.
- اختبار الاستعادة من النسخ الاحتياطية.
- dev/staging/prod فصل.
- System configuration + Testing. [web:133][web:135] مصدر إضافي

## 18.3 قواعد DevOps

- كل نشر يجب أن يكون قابلاً للتراجع.
- staging كل تغيير إنتاجي يجب أن يمر عبر.
- integration مراقبة كل.
- runbook كل مهمة حرجية يجب أن تمتلك.
- failure must be logged and triaged كل.
- System configuration + Testing. [web:133][web:135] مصدر إضافي

## 19. API وArcGIS References

### 19.1 Odoo API References

- [Odoo External JSON-2 API](#) [web:73]
- [Odoo Web Controllers](#) [web:81]
- [Odoo Security](#) [web:16]
- [Owl on Portal/Website](#) [web:89]

### 19.2 ArcGIS References

- [ArcGIS Maps SDK for JavaScript](#) [web:142]
- [ArcGIS Get Started](#) [web:143]
- [ArcGIS Sample Code](#) [web:148]
- [ArcGIS References](#) [web:146]

## 19.3 OCA GIS References

- [OCA/geospatial](#) [web:106][web:108]
- [Leaflet Map View \(OpenStreetMap\)](#) [web:153]

## 19.4 Integration rule

أو الخرائط يجب ألا يُنفذ بعشوائية. إذا وُجد أكثر من أسلوب APIs أي قرار تقني في

- اعرض كل أسلوب.
  - وضح المميزات.
  - وضح العيوب.
  - وضح التعارضات مع الموديولات الأخرى إن وجدت.
  - وضح أثره على الأمان والأداء والترقية.
  - بأقل مخاطرة وأعلى قابلية للصيانة Odoo اختر ما يخدم.
- مصدر إضافي: Testing + Security + Upgrade scripts + ArcGIS samples. [web:135][web:16]  
[web:92][web:148]

## 20. الخلاصة

الخير ليس مجرد مجموعة مبرمجين، بل نظام عمل هندسي متكامل. هذا الملف يحدد كيف يبني Odoo فريق قوية، آمنة، قابلة للصيانة، وقابلة للترقية عبر جميع الإصدارات المستهدفة، مع احترام الحوكمة Odoo الفريق حلول والورك فلو الداخلي للشركات، والعمل المنظم، ورفع التحديات أولاً بأول، وعدم التنفيذ بعشوائية ISO وال. مصدر إضافي: Developer docs + Security + Testing + System configuration. [web:1][web:16]  
[web:135][web:133]



1. <https://www.odoo.com/documentation/19.0/developer/reference/backend/security.html>
2. <https://developers.arcgis.com/javascript/latest/sdk-downloads-and-archive/>
3. <https://github.com/OCA/geospatial>
4. <https://developers.arcgis.com/javascript/latest/downloads/>
5. <https://developers.arcgis.com/javascript/latest/references/>
6. <https://github.com/OCA/geospatial/blob/16.0/README.md>
7. [https://apps.odoo.com/apps/modules/9.0/base\\_geoengine](https://apps.odoo.com/apps/modules/9.0/base_geoengine)
8. <https://oca.github.io/geospatial/>
9. <https://developers.arcgis.com/javascript/latest/downloads-and-previous-versions/>
10. <https://github.com/t-core-one/oca-geospatial>